

Práctica guiada microservicios con RabbitMQ

(Docker) microservicios en Node.js:

- **Producer:** API REST que registra personas (nombre, email, celular) y publica un mensaje en una cola.
- **Consumer:** worker que lee la cola y **simula** el envío de correo.

1) Prerrequisitos

- Docker instalado y funcionando.
- Node.js (>=14 recommended) y npm.
- Conocimientos básicos de terminal.

2) Levantar RabbitMQ con Docker

Ejecuta en tu terminal:

```
docker run -d --hostname my-rabbit --name rabbitmq \
-p 5672:5672 -p 15672:15672 \
rabbitmq:3-management
```

- Admin UI: <http://localhost:15672> (usuario guest / guest por defecto).
- AMQP: amqp://localhost:5672 (o amqp://guest:guest@localhost:5672).

Alternativa con docker-compose.yml (sencillo):

```
version: '3.8'
```

```
services:
```

```
  rabbitmq:
```

```
    image: rabbitmq:3-management
```

```
    container_name: rabbitmq
```

```
    ports:
```

```
      - "5672:5672"
```

```
      - "15672:15672"
```

```
    environment:
```

```
      RABBITMQ_DEFAULT_USER: guest
```

```
      RABBITMQ_DEFAULT_PASS: guest
```

docker-compose up -d

3) Estructura del proyecto (sugerida)

Crea dos carpetas paralelas:

micro-practica/

producer/ <-- microservicio registro (API)

consumer/ <-- worker que procesa la cola (simula email)

4) Variables de entorno comunes

Crea un archivo .env en ambos proyectos (producer y consumer):

RABBITMQ_URL=amqp://guest:guest@localhost:5672

QUEUE_NAME=email_queue

PORT=3000 # sólo en producer

5) Producer — API que registra y publica en la cola

5.1 Inicializar

Dentro de producer/:

mkdir producer && cd producer

npm init -y

npm install express amqplib dotenv body-parser

opcional: npm install nodemon --save-dev

5.2 package.json (ejemplo scripts)

```
{  
  "name": "producer",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"
```

```

},
"dependencies": {
  "amqplib": "^0.10.3",
  "body-parser": "^1.20.0",
  "dotenv": "^16.0.0",
  "express": "^4.18.0"
}
}

```

5.3 index.js (código completo, listo para ejecutar)

```

require('dotenv').config();

const express = require('express');
const bodyParser = require('body-parser');
const amqp = require('amqplib');

const RABBITMQ_URL = process.env.RABBITMQ_URL || 'amqp://localhost';
const QUEUE = process.env.QUEUE_NAME || 'email_queue';
const PORT = process.env.PORT || 3000;

let channel;

async function initRabbit() {
  try {
    const conn = await amqp.connect(RABBITMQ_URL);
    channel = await conn.createChannel();
    await channel.assertQueue(QUEUE, { durable: true }); // cola durable
    console.log('✅ Conectado a RabbitMQ en', RABBITMQ_URL, 'cola:', QUEUE);

    // cerrar conexión al terminar el proceso
    process.on('SIGINT', async () => {

```

```
    try { await channel.close(); await conn.close(); } catch(e) {}

    process.exit(0);

  });
} catch (err) {

  console.error('Error conectando a RabbitMQ', err);

  process.exit(1);

}
}
```

```
async function startServer() {
```

```
  await initRabbit();
```

```
  const app = express();
```

```
  app.use(bodyParser.json());
```

```
  // almacenamiento en memoria (sólo para demo)
```

```
  const users = [];
```

```
  // endpoint registro
```

```
  app.post('/register', async (req, res) => {
```

```
    const { name, email, cell } = req.body;
```

```
    if (!name || !email || !cell) {
```

```
      return res.status(400).json({ error: 'Faltan campos: name, email, cell' });
```

```
    }
```

```
    const user = {
```

```
      id: Date.now(),
```

```
      name,
```

```
      email,
```

```

    cell,

    createdAt: new Date().toISOString()
  };

  users.push(user);

  const payload = { type: 'NEW_USER', user };

  try {
    const sent = channel.sendToQueue(Queue,
    Buffer.from(JSON.stringify(payload)), {
      persistent: true // marca persistente para sobrevivir reinicios
    });

    console.log(' Mensaje enviado a la cola:', payload);

    return res.status(201).json({ ok: true, user });
  } catch (err) {
    console.error(' Error publicando en la cola', err);

    return res.status(500).json({ error: 'No se pudo enviar a la cola' });
  }
});

app.get('/users', (req, res) => res.json(users));

app.listen(PORT, () => {
  console.log(` Producer API en http://localhost:${PORT} `);
});

startServer().catch(err => {
  console.error('Fallo al iniciar producer', err);
});

```

```
process.exit(1);  
});
```

6) Consumer — lee la cola y simula envío de correo

6.1 Inicializar

Dentro de consumer/:

```
mkdir consumer && cd consumer
```

```
npm init -y
```

```
npm install amqplib dotenv
```

```
# opcional: npm install nodemon --save-dev
```

6.2 package.json (ejemplo)

```
{  
  "name": "consumer",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  },  
  "dependencies": {  
    "amqplib": "^0.10.3",  
    "dotenv": "^16.0.0"  
  }  
}
```

6.3 index.js (código completo)

```
require('dotenv').config();
```

```
const amqp = require('amqplib');
```

```
const RABBITMQ_URL = process.env.RABBITMQ_URL || 'amqp://localhost';
```

```
const QUEUE = process.env.QUEUE_NAME || 'email_queue';
```

```
async function simulateEmailSending(user) {  
  // Aquí simulas la lógica de envío de correo.  
  console.log(` Simulando envío de correo a: ${user.email} (nombre: ${user.name}) `);  
  // Simula latencia de envío  
  await new Promise(r => setTimeout(r, 1500));  
  console.log(` Correo "enviado" a ${user.email} (simulado) `);  
}
```

```
async function startConsumer() {
  try {
    const conn = await amqp.connect(RABBITMQ_URL);
    const channel = await conn.createChannel();
    await channel.assertQueue(Queue, { durable: true });
    // Recomendando prefetch(1) para distribuir carga entre consumidores
    channel.prefetch(1);

    console.log('Esperando mensajes en la cola:', Queue);

    channel.consume(Queue, async (msg) => {
      if (msg !== null) {
        try {
          const content = JSON.parse(msg.content.toString());
          console.log('Mensaje recibido:', content);

          if (content.type === 'NEW_USER' && content.user) {
            await simulateEmailSending(content.user);
            // ack (confirmamos que procesamos bien)
          }
        } catch (error) {
          console.error('Error al procesar mensaje:', error);
        }
      }
    });
  } catch (error) {
    console.error('Error al iniciar el consumidor:', error);
  }
}
```

```

        channel.ack(msg);
    } else {
        console.log(' Mensaje con tipo desconocido, ack y descartar');
        channel.ack(msg);
    }
} catch (err) {
    console.error(' Error procesando mensaje:', err);

    // si ocurre error se puede requeue o enviar a DLX; aquí requeue=false para no
    bloquear

    channel.nack(msg, false, false);
}
}
}, { noAck: false });
} catch (err) {
    console.error(' Error en consumer', err);
    process.exit(1);
}
}

startConsumer();

```

7) Probar la práctica

1. Asegúrate RabbitMQ levantado (ver paso 2).
2. En producer/: npm start (o npm run dev).
3. En consumer/: npm start (o npm run dev).
4. Registrar un usuario (ejemplo curl):

curl -X POST http://localhost:3000/register \

-H "Content-Type: application/json" \

-d '{"name":"Juan Perez","email":"juan.perez@example.com","cell":"78000000"}'

- Respuesta: JSON con user.

- Producir imprimirá en consola el envío del mensaje.
- Consumer mostrará que recibió el mensaje y “envió” el correo.

En la UI de RabbitMQ (<http://localhost:15672>) puedes ver la cola email_queue, número de mensajes, consumers, etc.

8) Notas técnicas y buenas prácticas (pequeña guía)

- **Durable queue + persistent messages:** lo configuramos para que mensajes sobrevivan reinicios del broker.
- **Ack / nack:** el consumidor confirma (ack) sólo cuando procesó correctamente.
- **Prefetch(1):** evita que un consumidor reciba múltiples mensajes simultáneamente si no los ha procesado.
- **Retry / Dead-letter:** para producción agrega DLX (dead-letter exchange) y reintentos limitados.
- **Seguridad:** NO uses guest/guest en producción, habilita usuarios con permisos mínimos, TLS, etc.
- **Escalado:** puedes tener múltiples instancias del consumer para paralelizar envíos.
- **Persistencia real:** guarda los usuarios en DB (Postgres, Mongo) en vez de memoria.
- **Envío real de email:** sustituir simulateEmailSending por nodemailer con un SMTP (o servicios como SendGrid, SES).

9) Extensiones sugeridas (para practicar)

- Integrar **nodemailer** y enviar correos reales (usa cuentas de prueba como Ethereum o un SMTP).
- Dockerizar los microservicios y usar docker-compose para levantar todo (rabbitmq + producer + consumer).
- Añadir endpoint para consultar estado del envío (usar base de datos y tabla de email_jobs).
- Implementar reintentos exponenciales y DLX.

10) Resumen rápido de comandos útiles

1) Levantar rabbit

```
docker run -d --hostname my-rabbit --name rabbitmq -p 5672:5672 -p  
15672:15672 rabbitmq:3-management
```

2) Producer

```
cd producer
```

```
npm install
```

```
npm start
```

3) Consumer

```
cd ../consumer
```

```
npm install
```

```
npm start
```

4) Probar

```
curl -X POST http://localhost:3000/register \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"name":"Prueba","email":"prueba@example.com","cell":"700000000"}'
```

**Ejercicio Dockerizar todos los microservicios y generar un archivo yml para
levantar todos los servicios juntos**

: